

Claude Code, 이제 팀 도구로 쓰기

하네스 설정으로 달라지는 것들

발표자
sumin@iotrust.kr

일시
2026

오늘 다룰 내용

왜 → 무엇 → 어떻게 → 어떻게 쓰는지

01	왜 필요한가	지금 Claude를 쓰면서 겪는 불편함	03
02	하네스란	진화 → 본질 정의 → 좁은/넓은 의미	04-06
03	적용하면 달라지는 것	Before / After	07
04	프로젝트 적용	폴더 분리 + 4단계 설치	08-09
05	구성 한눈에	구조 트리 + 강도 · 플러그인	10-11
06	6개 구성요소	CLAUDE.md · rules · hooks · skills · agents · plugin	12-22
07	활용법 4가지	같이 계획 · 검증 · 자동화 · 규칙화	23-26
08	팀 맞춤화	시작 철학 + 실행 가이드	27-28
09	일상 워크플로우	작은 / 큰 / 다관점 심의	29
10	FAQ · 마무리 · 예고	자주 묻는 질문 + 다음 시간	30-32

지금 어떻게 쓰고 계세요?

익숙한 풍경 — 그리고 그 안의 불편함

💬 이런 경험 있으시지 않나요?

이 파일은 건드리지 마.

알겠습니다.

... 잠시 후 ...

어? 화면이 깨졌네.

죄송합니다, 관련 파일을 같이 수정했어요.

대화에서 말한 규칙은
세션이 끝나면 사라집니다.

다음 날 새 대화를 열면 — 처음부터 다시 설명.

팀원마다 다른 커밋 메시지

A는 영어 / B는 한국어

팀원마다 다른 저장 방식

A는 main 바로 / B는 분리

매번 같은 설명 반복

"이 프로젝트는..." 처음부터

"똑똑한 질문" vs "잘 일하는 환경"

AI와 일하는 방식은 3단계로 진화해왔어요



오늘 다룰 건 **Level 3** — AI가 잘 일할 수 있는 환경을 만드는 이야기

하네스란 무엇인가

사람이 매번 안 말해도, AI가 같은 결과를 내는 작업 환경 전체

하네스(harness) — 말 안장의 가죽끈처럼, AI가 엉뚱한 데로 가지 않게 잡아주는 장치



맥락

뭘 하는 곳?

CLAUDE.md



규칙

뭘 하면 안 되나?

rules/



도구

자주 쓰는 절차는?

skills/ · agents/



검증

잘 했는지 누가 확인?

hooks/ · /qa · /wrap

프롬프트 한 줄도, `.claude` 폴더 하나도 아닙니다 — 4가지가 한 묶음

하네스의 정의는 생각보다 넓다

`.claude` 폴더 만이 아니다

좁은 의미

CLAUDE.md +
`.claude/` 폴더

넓은 의미 — 오늘 다룰 것

폴더 구조
+ 문서 분리
+ `.claude/`
+ 플러그인

맥락

CLAUDE.md · docs/

규칙

`.claude/rules/`

도구

skills/ · agents/ · plugin

검증

hooks/ · /qa · /wrap

흔적

`.dev/` (AI 메모)

하네스를 적용하면

같은 프로젝트를 쓰는 사람 모두가 동일한 규칙으로 시작

적용 전		적용 후
팀원마다 작업 방식이 제각각	→	하나의 "하네스"로 규칙 통일
Claude 답변이 매번 달라짐	→	정해진 규칙 안에서 일관된 답변
실수로 main 브랜치에 올릴 위험	→	시스템이 자동으로 막아줌
잘못해도 모르고 지나감	→	자동으로 감지해서 알려줌
새 팀원이 오면 처음부터 설명	→	폴더만 복사하면 바로 동일 환경

사람 서랍 vs AI 서랍

프로젝트 폴더를 처음부터 사람용 / AI용 으로 나누세요

내 프로젝트/

|— src/ # 작업 대상

|— docs/ # 사람

|— .dev/ # AI

|— .claude/ # 사람 + AI

|— CLAUDE.md # 지도

사람 docs/

회사 규칙 · 브랜드 톤 · 체크리스트 · 공식 양식

AI .dev/

작업 중 알게 된 주의점 · 실험 기록 · 디버깅 흔적

사람 + AI .claude/

규칙 · 혹 · 스킬 · 에이전트 설정

왜 분리? 섞이면 3개월 뒤 AI가 뭘 봐야 할지 헷갈려서 **품질이 떨어집니다**

내 프로젝트에 적용하기

큰 번호 4단계 — 복사 · 폴더 · 플러그인 · 권한

PRE

Slack에 올린 하네스 폴더 다운받기

1

파일 복사

CLAUDE.md + .claude/ 를 프로젝트 맨 위에 붙여넣기

2

폴더 생성

docs/ 사람용 공식 문서
.dev/ AI 작업 메모
src/ 실제 코드

3

플러그인 설치

```
/plugin install harness-session  
@harness-local  
/plugin install session-wrap@te  
am-attention-plugins
```

4

실행 권한

```
chmod +x .claude/hooks/guard.sh
```

한 번만

하네스 구조 한눈에 보기

4개 폴더 + 1개 파일 — 이게 전부

프로젝트 루트/

└─ **CLAUDE.md** ← 핵심 지시 파일 (가장 먼저 읽힘)

└─ **docs/** ← 사람이 관리 — 비즈니스의 진실

└─ **.dev/** ← AI가 남기는 작업 흔적

└─ **src/** ← 실제 코드

└─ **.claude/**

└─ **rules/** ← 행동 규칙

└─ **hooks/** ← 자동 안전장치

└─ **skills/** ← 자주 쓰는 절차 모음

└─ **agents/** ← 전문 도우미

+ **플러그인** (탐색기엔 안 보이지만 사용 가능)

harness-session · session-wrap

구성요소별 영향 강도

같은 하네스 안에서도 작동 강도가 다릅니다

CLAUDE.md

새 대화 시작 시 자동 읽힘



낮음

rules/

모든 대화에 항상 적용



중간

hooks/

작업할 때마다 자동 차단·개입



가장 강함

skills/

/이름 호출 + 자동



낮음

agents/

큰 요청 시 자동 호출



중간

plugin

위 요소를 묶어 한 번에



중간

PLUGIN

플러그인은 `~/.claude/plugins/cache/` 에 저장 — 탐색기엔 안 보임. `/plugin install` 로 각자 설치.

CLAUDE.md

가장 먼저 읽히는 핵심 지시 파일

프로젝트 작업 규칙

> 이 프로젝트는 *Claude Code* 하네스 보일러플레이트입니다.

세션 시작 시

새 대화가 시작되면 반드시 `/start` 를 실행한다.

작업 흐름 (순서 엄수)

1. 기능 구현
2. 로컬 확인 (사용자 확인 요청)
3. 커밋 & 푸시

큰 기능 요청 시

`planner` → `designer` → `frontend` → `backend`

세부 규칙 참조

`.claude/rules/` `git.md` · `communication.md` · `security.md`

자동으로 먼저 읽히는 종이 + 다른 규칙 파일을 함께 데려오는 목차 역할

CLAUDE.md, 왜 핵심인가

3가지 특징

01

자동으로 읽힘

새 대화를 열 때마다 Claude가 가장 먼저 보는 종
이. 따로 명령 없이 항상 로드.

02

규칙의 출발점

여기서 `rules/` 안 자세한 규칙들을 불러옴. 모든
규칙은 이 파일에서 시작.

03

경제적 — 토큰 절약

매번 대화로 설명하던 걸 한 번에 — 단 너무 길면
오히려 성능 저하. **약 200줄 이내** 유지, 세부 규칙
은 `rules/` 로 분리.

주의

CLAUDE.md는 **권고 사항** 이라 Claude가 무시할 수 있어요 — 반드시 지키게 하려면 `.claude/hooks/` 로 강제하세요

rules — 3가지 규칙집

Claude가 일관되게 행동하도록 정해 둔 규칙 모음

소통 방식

communication.md

- 에러 → 쉬운 말로 풀이
- 영향 범위 먼저 알림
- 요청 애매하면 질문 먼저
- 완료 시 결과 요약

저장 · 배포

git.md

- 변경 기록(커밋) 메시지 형식 통일
- 작업 폴더(브랜치) 이름 규칙
- 최종본(main)에 직접 올리기 금지
- 리뷰 후 깔끔하게 합치기

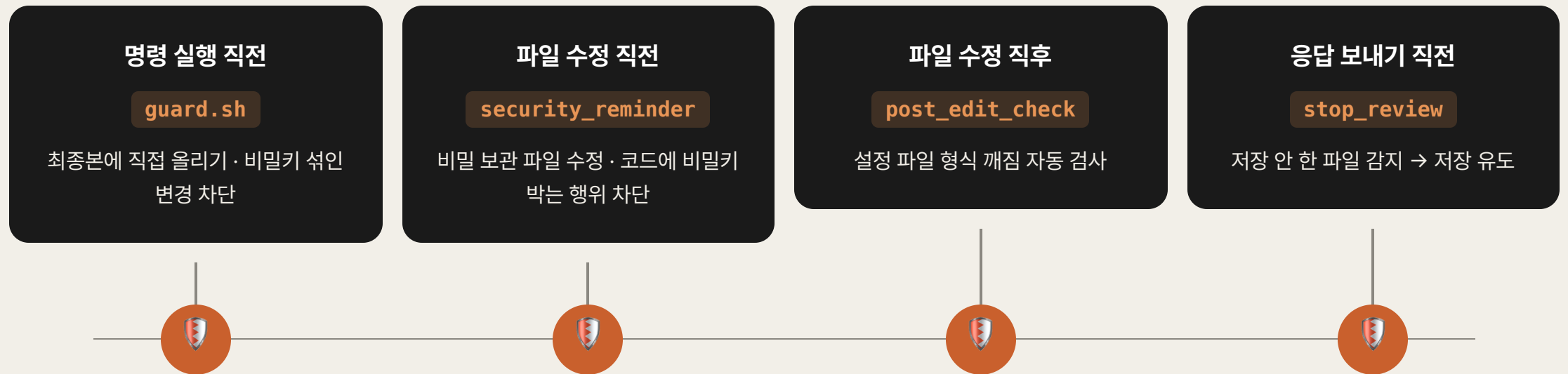
비밀값 · 인증

security.md

- 비밀키 값(예: 구글 로그인 키)은 비밀 보관 파일에 직접
- 안전한 로그인 방식 권장
- 오래된 보안이 약한 토큰은 새로 만들지 않기
- 비밀값이 노출되면 즉시 비활성화

hooks

Claude가 작업하기 전·중·후에 자동으로 끼어드는 안전장치



skills — 자주 쓰는 절차 묶음

두 가지 출처 — 하나는 외부, 하나는 우리 팀이 만든 것

EXTERNAL

외부 스킬

출처 Anthropic / Supabase · 커뮤니티

설치 `npx skillsadd` · 폴더 복사

수정  업데이트 충돌 방지

예시 `frontend-design`

SELF-MADE

자체 제작 스킬

출처 우리 팀이 직접 작성

설치 `SKILL.md` 추가

수정  자유롭게 편집

예시 `start` · `brainstorming`

현재 설치된 스킬 6개

/이름 으로 직접 호출 + Claude가 자동 발동

팀 제작

start

세션 시작 5단계 체크

새 대화 / `/start`

팀 제작

brainstorming

아이디어 확산 + 심층 분석

"아이디어 내줘"

팀 제작

doc-edit

마크다운 구조 유지 편집

"README 수정"

ANTHROPIC

frontend-design

일관된 UI 가이드

"UI 만들어줘"

ANTHROPIC

canvas-design

포스터 · PNG · PDF 비주얼

"포스터 만들어줘"

SUPABASE

supabase-postgres-best-practices

DB 쿼리 · 스키마 최적화

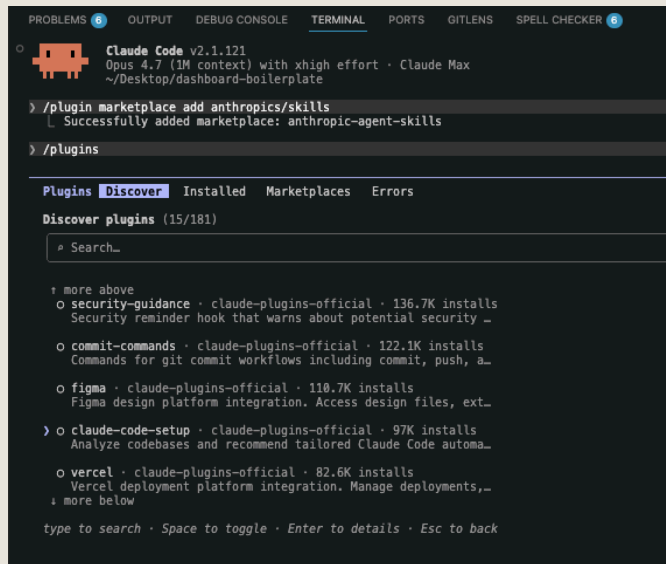
"쿼리 튜닝"

외부 스킬은 어떻게 받나

CLI 명령으로 받거나, VS Code 확장 UI 에서 클릭으로 설치

방법 1 CLI 명령으로 설치

1. `/plugin marketplace add anthropics/skills` — 마켓 등록
2. `/plugins` 입력 → Discover 탭
3. 스킬 선택 → Enter 로 설치



```
CLAUDE CODE v2.1.121
Opus 4.7 (1M context) with xhigh effort · Claude Max
~/Desktop/dashboard-boilerplate

> /plugin marketplace add anthropics/skills
  Successfully added marketplace: anthropic-agent-skills

> /plugins

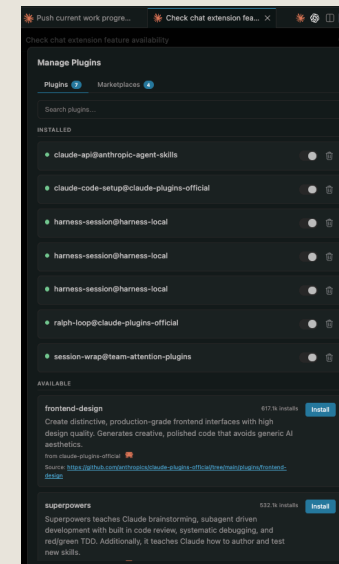
Plugins | Discover | Installed | Marketplaces | Errors
Discover plugins (15/181)
  Search...

  more above
  o security-guidance · claude-plugins-official · 136.7K installs
    Security reminder hook that warns about potential security ...
  o commit-commands · claude-plugins-official · 122.1K installs
    Commands for git commit workflows including commit, push, a...
  o figma · claude-plugins-official · 110.7K installs
    Figma design platform integration. Access design files, ext...
  o claude-code-setup · claude-plugins-official · 97K installs
    Analyze codebases and recommend tailored Claude Code automa...
  o vercel · claude-plugins-official · 82.6K installs
    Vercel deployment platform integration. Manage deployments,...
  more below

type to search · Space to toggle · Enter to details · Esc to back
```

방법 2 VS Code 확장 UI

1. Claude 채팅창에 `/Manage Plugins` 입력
2. Available 목록에서 스킬 찾기
3. Install 버튼 클릭



설치 과정과 결과

설치한 스킬은 `~/.claude/plugins/cache/.../skills/<스킬이름>/SKILL.md` 로 자동 추가 — 이후 별도 명령 없이 인식됩니다

VIDEO CLI 설치 전체 흐름



영상 첨부 위치 — `install_skills_sh.mov`

RESULT 설치 후 — 자동 인식

```
~/.claude/plugins/cache/  
├ <마켓이름>/  
  └ <플러그인>/skills/  
    └ <스킬이름>/SKILL.md
```

→ Claude가 자동 인식 · 별도 등록 불필요

우리 레포의 에이전트 6명

"무조건 체인"이 아니라 "상황에 맞는 패턴 1개"

planner

PRD · 사용자 스토리 · 화면 흐름

비즈니스 우선순위 · 기능 범위

designer

와이어프레임 · 비주얼 컨셉

UX 흐름 · 정보 구조 · 접근성

frontend

React/Next.js UI 코드

구현 현실성 · 성능 · 재사용성

backend

DB 스키마 · API · RLS

데이터 설계 · 쿼리 성능 · 보안

code-architect

기존 패턴 분석 → 설계도

아키텍처 일관성 · 의존성 방향

idea-researcher

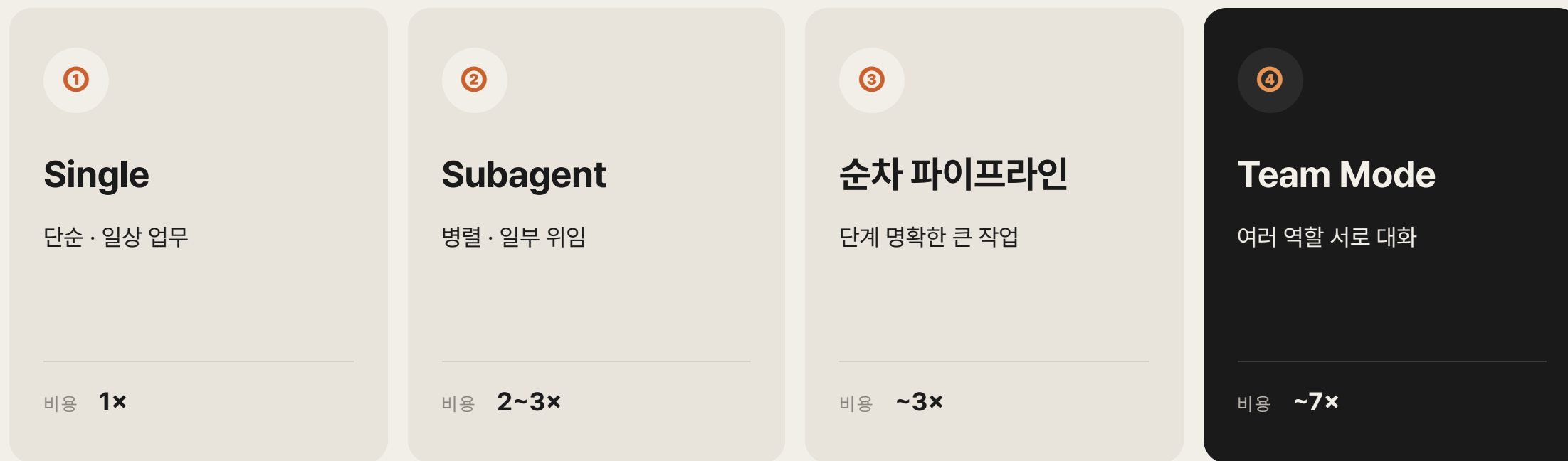
아이디어 심층 분석

brainstorming 전용 서브

기본 흐름 — `planner → designer → frontend → backend` · 메인 Claude가 종합

agents — 4가지 분업 패턴

복잡한 작업을 Claude가 알아서 분담



대부분은 ① ~ ② 로 끝납니다. ④ Team Mode 는 토큰 비용이 **7배** 라 꼭 필요할 때만.

Plugin

스킬 · 훅 · 에이전트를 한 묶음으로 — 통째로 받기

REQUIRED

현재 레포 의존성 — `/plugin install` 로 설치

harness-session

초기 세팅 · 요구사항 명확화 · 검증 · 오케스트레이션 (총 7개 스킬)

`/scaffold`

`/specify`

`/qa`

`/check-harness`

session-wrap

세션 회고 · 반복 패턴 자동 감지

`/wrap`

`/history-insight`

`/session-analyzer`

OPTIONAL

권고 — 살펴보면 좋은 외부 플러그인

superpowers

발산→계획→실행 흐름 + TDD·디버깅 스킬

`/superpowers:brainstorm`

`:write-plan`

`:execute-plan`

INSTALL

```
/plugin install superpowers@claude-plugins-official
```

gstack

23개 역할 기반 스킬 (CEO·디자이너·엔지니어·QA...)

`/autoplan`

`/design-shotgun`

`/ship`

`/retro`

INSTALL

```
git clone --single-branch --depth 1 https://github.com/garrytan/gstack.git ~/.claude/skills/gstack && cd ~/.claude/skills/gstack && ./setup
```

활용 1 — 같이 계획부터

"해줘"가 아니라 "물어봐"

❌ ANTI-PATTERN

"해줘"

"이거 만들어줘" → AI가 알아서 만들
→ "아닌데..." → 다시 시킴
→ 시간 낭비



✅ 좋은 패턴

"같이 계획부터"

"~~할 건데 같이 계획 세우자"
→ 사람이 검토·수정·승인
→ 높은 성공률

나

/specify 결제 페이지 만들어줘

AI

계획을 세우겠습니다. 작업 전 3가지 확인:

1. 결제 수단은 카드만? 카카오페이/네이버페이도?
2. 비회원 결제 허용?
3. 결제 실패 시 재시도는 자동 / 수동?

활용 2 — 만든 AI ≠ 평가하는 AI

"자기가 만든 걸 자기가 평가하면, 그저 그런 결과도 자신있게 칭찬한다"

GENERATOR

만드는 AI

관대함

"이 정도면 괜찮아"

≠

EVALUATOR

평가하는 AI

회의적으로 튜닝

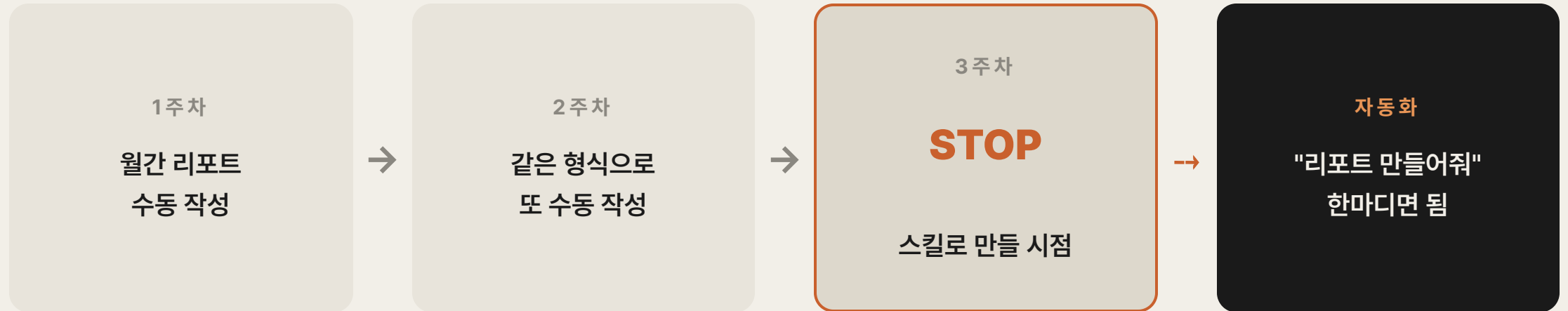
"이거 정말 맞나?"

⚠ 같은 대화창에서 만들고 평가하지 말 것 — **관점을 물리적으로 분리**

실전: 결과물 만든 세션을 `/clear` 하고 새 세션에서 비판적 검토 · 또는 `/qa` 스킬

활용 3 — 3번 반복 → 스킬

같은 작업을 3번째 하게 되면, 그때가 자동화할 타이밍



💡 세션 끝에 `/wrap` 실행 → 반복 패턴 자동 감지 → 스킬 초안 제안 → 승인만 하면 생성

활용 4 — 3번 틀림 → 규칙

같은 실수를 3번째 하면, 규칙이 빠진 것

1차 — 자동

auto-memory

교정 발언 ("아니", "하지 마")

→ `memory/feedback_*.md` 자동 저장

→ 다음 대화 시작 시 자동 재로드

2차 — 반영

rules/ 에 못박기

같은 실수 3번째면

일회성 기억을 넘어 **규칙 파일에 명시**

→ 팀 전체에 자동 적용

Rules 에 못박는 명령 예시

USER 발언

"이거 앞으로 `.claude/rules/git.md` 에 추가해줘
— 커밋 메시지에 `Co-Authored-By` 넣지 말 것"



CLAUDE 동작

해당 파일에 규칙 한 줄 추가 + 커밋 → 팀 전체에 자동 적용

가볍게 시작, 자주 수정

하네스는 한 번에 완성되지 않습니다

느슨하게 시작



써본다



불편한 부분 발견



한 줄 고친다



좋은 하네스는 점점 단순해진다 — 복잡해지고 있다면 뭔가 잘못된 것

✅ 잘 가고 있는 신호

- 같은 지적을 두 번 안 해도 됨
- 실수가 규칙이 된다 (자동 축적)
- 차단 장치가 뭔가를 막고 있다
- 불필요한 규칙이 줄어든다

⚠️ 막힌 신호

- 확인에 시간이 더 오래 걸린다
- 시켰는데 원하는 결과가 안 나온다
- 규칙·도구가 많은데 잘 안 쓴다
- 설명서가 길어지고 관리가 안 된다

내 팀에 맞게 고치는 법

규칙 파일은 전부 텍스트 — 누구나 수정 가능

먼저 이 3개부터

도입 첫 주에 손볼 핵심

CLAUDE.md

루트

프로젝트 설명 · 배포 환경/머지 흐름 주석 해제

git.md

RULES

브랜치 이름 규칙 · 커밋 언어 · 배포 방식

security.md

RULES

팀 비밀값 관리 도구 (1Password / Vault 등)

communication.md

RULES

팀 용어 · 노션/피그마 링크

guard.sh

HOOKS

추가 차단 명령어 · 비밀값 패턴

start/SKILL.md

SKILLS

시작 전 체크사항 (예: Slack)

planner.md

AGENTS

에이전트 페르소나 · 출력 형식

⚠ 외부 스킬(frontend-design 등)은 업데이트 충돌 방지 위해 **직접 수정 X**. 팀 제작 스킬(start , brainstorming)은 자유롭게.

일상 워크플로우

하네스 적용 후 — 세 가지 시나리오

SMALL

작은 요청

버튼 색 · SQL 한 줄

Single 패턴

- 1 /start
- 2 요청
- 3 Claude 작업
- 4 로컬 확인
- 5 "OK"
- 6 자동 저장

A

BIG

큰 요청

대시보드 · 신규 모듈

순차 파이프라인

- 1 /start
- 2 /specify
- 3 planner→designer→frontend→backend
- 4 단계별 검토
- 5 /qa
- 6 /wrap

B

TEAM

다관점 심의

기획·구현·아키텍처 다같이

Team Mode

- 1 /start + /specify
- 2 /agent-orchestrate
- 3 5명 에이전트 심의
- 4 메인 Claude 종합
- 5 /qa

C

▲ 팀 모드는 토큰 비용 ~7배 · 대부분은 B로 충분

자주 묻는 질문

비개발자가 자주 묻는 4가지

Q. 기존 프로젝트에도 적용할 수 있나요?

네, 새 프로젝트가 아니어도 됩니다. 이 레포의 `.claude/` 폴더와 `CLAUDE.md` 파일을 그대로 복사해 넣으세요. 그 다음 **VS CODE · CLAUDE 채팅창** 에 `/plugin install harness-session@harness-local` 두 줄만 한 번씩 입력하면 끝나요.

Q. 플러그인이 탐색기에 왜 안 보여요?

플러그인은 프로젝트 폴더가 아니라 사용자 홈(`~/.claude/`)에 캐시 됩니다. 그래서 탐색기에 파일이 없어 보여도 정상이에요. **VS CODE · CLAUDE 채팅창** 에서 `/plugin install <이름>` 한 번 실행 → 이후엔 자동 인식됩니다.

Q. 한 번 세팅하면 팀원 모두에게 자동 적용?

코드처럼 git 으로 push 하면 팀 전체에 반영됩니다. 단, **플러그인은 각자 한 번씩 설치**해야 해요. **VS CODE · CLAUDE 채팅창** 에서 `/plugin install ...` 명령을 한 번 실행하면 그 PC에선 계속 동작합니다.

Q. 규칙에 없는 내용은 어떻게 동작하나요?

Claude Code 가 평소처럼 알아서 답하고 실행합니다. 별도로 강제하는 규칙이 없으면 일반 동작 그대로예요. 자주 어긋나는 부분이 있으면 **VS CODE · CLAUDE 채팅창** 에서 `/wrap` 으로 규칙화하면 됩니다.

마무리

오늘 꼭 가져가실 3가지

01

범 위

.claude 만이
하네스가 아닙니다

폴더 구조 분리에서부터 시작.

docs · .dev · src · .claude

02

운 영

가볍게 시작,
자주 수정

처음부터 완벽하게 만들지 마세요.

잘 굴러가는 하네스는 시간이 갈수록 단순해집니다.

03

루 틴

/start → 작업 → /wrap

시작은 **/start**, 마무리는 **/wrap** 으로
한 사이클을 닫아 주세요.

오늘 끝나고

하네스 폴더 복사 → 플러그인 설치 → 다음 작업 때 **/start** 한 번

NEXT

다음 시간 예고

2회차 — 자체 프로젝트에 스킬·에이전트 적용 실습

01

프로젝트 적용

보유 레포에 하네스 설치 후 `/start` 실행

02

스킬·에이전트 활용

설치된 스킬·에이전트로 실작업 수행

03

워크플로우 운용

컨텍스트 관리 · 자동화 루프 · 팀 모드

04 · 상시 정비

규칙화 · hook 설정

반복되는 실수는 규칙으로 명문화 · 반복되는 절차는 hook 으로 자동화

문의 · sumin@iotrust.kr Thank you.